Expert Systems With Applications 248 (2024) 123403

Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa

RAKCR: Reviews sentiment-aware based knowledge graph convolutional networks for Personalized Recommendation

Yachao Cui [a,b], Hongli Yu [a], Xiaoxu Guo [a,c], Han Cao [a,*], Lei Wang [a]

[a] School of Computer Science, Shaanxi Normal University, Xi'an, 710000, China
[b] Department of Computer Technology and Applications, Qinghai University, Xining, 810016, China
[c] School of Artificial Intelligence Institute, Beijing Economic Management Vocational College, Beijing, 100018, China

ARTICLE INFO

Keywords:
Text Sentiment Analysis
Knowledge graph
Graph neural network
Personalized recommendation

ABSTRACT

The recommendation algorithm is an important means to alleviate the information explosion in the era of big data. There has been a great deal of research into the use of knowledge graphs as auxiliary information in recommender systems, which can be used to alleviate data sparsity and cold start problems. However, most knowledge graph-based recommendation methods only use rating data to capture the user's potential interest, and the rating is only a comprehensive evaluation of the item by the user, which cannot intuitively and accurately express the user's personalized preference. In addition, existing recommendation strategies that blend ratings and reviews cannot simultaneously model the aspect fine-grained sentiment preferences of users in reviews as well as the personalized characteristics of items from the user's perspective. To this end, in this paper, we propose Reviews Sentiment-Aware Knowledge Graph Convolutional Neural Network (RAKCR), a generic review and knowledge graph-based framework that provides better recommendations by fully mining the fine-grained personalization features in user reviews. In contrast to existing correlation recommendation methods, we designed a new reviews sentiment perception feature and knowledge graph alignment module to characterize user preferences for specific features of items in the knowledge graph. To better represent the personalized feature distribution of users and items, we use the proposed RAKCR to aggregate sentiment relationship weight-aware neighborhood information in the knowledge graph to capture personalized feature representations of both users and items, and to better learn user and item embeddings for more accurate personalized recommendations. Experimental results demonstrate that the proposed RAKCR model outperforms the benchmark model significantly in click-through rate prediction for recommendation scenarios. Across the three datasets, Movielens-20 m, Amazon-book, and Yelp, the AUC values show an average improvement of 6.4%, 6.0%, and 3.4%, respectively. Additionally, the F1 values exhibit an average improvement of 7.2%, 6.2%, and 4.1%, respectively, when compared to existing state-of-the-art methods.

1. Introduction

With the rapid development of the Internet, the amount of information in various applications has shown explosive growth. How to accurately filter out information that meets the user's personalized interests has always been a problem that recommendation systems focus on and solve. The Collaborative filtering algorithm is a traditional and effective recommendation strategy in recommendation systems, but there are interaction sparsity and cold start problems (Koren, 2008; Sarwar, Karypis, Konstan, & Riedl, 2001; Wang, He, Wang, Feng, Chua, & Tat-Seng, 2019). To this end, researchers usually extract features from various side information to improve the performance of recommender systems (Chen, He, & Kan, 2016; Cheng, Chang, Zhu, Kanjirathinkal, & Kankanhalli, 2019; Jamali & Ester, 2010).

Knowledge graph technology has been rapidly developed and used in many research fields in recent years. Knowledge graphs are also used as more effective side information to improve the performance and interpretability of recommendation systems because of their rich semantic associations between entities, and they have achieved great success. In some recent studies in the field of knowledge graph-based recommendation, most researchers only use ratings as user's historical data to propagate preferences in knowledge graphs and mine the potential interests of users. For example, RippleNet (Wang et al., 2018) proposes an end-to-end framework to iteratively propagate user

* Corresponding author.
E-mail addresses: cuiyachao@snnu.edu.cn (Y. Cui), yuhongli@snnu.edu.cn (H. Yu), gxx123@snnu.edu.cn (X. Guo), caohan@snnu.edu.cn (H. Cao), wanglei521@snnu.edu.cn (L. Wang).

https://doi.org/10.1016/j.eswa.2024.123403
Received 13 June 2023; Received in revised form 26 January 2024; Accepted 2 February 2024
Available online 9 February 2024
0957-4174/© 2024 Elsevier Ltd. All rights reserved.

preferences in knowledge graphs and expand users' potential interests by exploiting user history-clicked items. The attention-enhanced knowledge-aware user preference model (AKUPM) (Tang, Wang, Yang, & Song, 2019) divides interactions between entities into inter-entity and intra-entity, and it uses attention mechanisms to explore the importance of different entities to user representations and filter irrelevant entities to better perceive user preferences for click-through rate prediction. Aiming at the problem of modeling each user interaction as an independent data instance in previous methods, KGAT (Wang, He, Cao, Liu, Chua, & Tat-Seng, 2019) uses user historical interaction data to model high-order relationships in a knowledge graph attention network to extract collaborative signals from collective behavior. To address KG's sparsity and noise problems, KGCL (Yang, Huang, Xia, & Li, 2022) designed a generic knowledge graph comparative learning framework to alleviate the information noise of knowledge graph-enhanced recommender systems. These methods use rating data in conjunction with knowledge graphs to mine users' potential interests and preferences, while rating data is only an overall rating of the item by the user and does not reflect the user's true personalized preferences. For example, if a user rates a movie with 4.5 points (out of 5 points), it does not mean that the user really likes the item, but may have thought the movie was generally good and just habitually gave good reviews, without being able to get more information from the rating. Mining the user's potential interests in the knowledge graph based on ratings is a viable solution. However, it does not provide an intuitive and accurate representation of the user's individual needs. Some researchers have jointly used ratings and reviews to mine users' personalized preferences in the knowledge graph. For example, Park, Chae, Bae, Park, and Kim (2022) constructed an emotion-aware knowledge graph by analyzing users' ratings and reviews of items, and used reinforcement learning strategies for item recommendation and reasoning. The literature (Liu, Miyazaki, & Chang, 2022) extracts entities from reviews, constructs a knowledge graph fusing user, item, and review entities, learns cross-features from reviews and ratings, and proposes a multi-task learning method based on ratings and reviews for recommendation and link prediction. These methods use recommendation strategies that combine ratings and reviews to better mine user preferences, but they cannot simultaneously model the user's fine-grained emotional preferences and user-perspective item personalization features. In addition, knowledge graphs are structured data, reviews are unstructured data, and fusing these two heterogeneous data into a new knowledge graph introduces noise, which can affect the recommendation results. Furthermore, these solutions incorporating review features are only for a single domain and not a general recommendation framework. We believe that the review data contains a large number of user comments on the details of the item, from which it is possible to uncover the individual needs of the user, and in particular the concerns of the user about the item. Through sentiment analysis of the review data, the sentiment of users towards specific item features can be accurately captured, as well as the distribution of sentiment of different users towards the same item, so that fine-grained personalized features of users based on aspect sentiment and personalized features of items perceived from the user's perspective sentiment can be constructed.

To this end, in this paper, we propose the Review Sentiment-Aware Knowledge Graph Convolutional Neural Network (RAKCR). It is a generic review and knowledge graph-based framework that provides personalized recommendations by fully exploiting the fine-grained personalized features in user reviews. Different from existing related recommendation methods, we design a new review sentiment-aware feature alignment module to the knowledge graph, which aligns sentiment features extracted from reviews with the adjacencies of the corresponding item entities in the knowledge graph to characterize user preferences for specific features of the item. With the aim of better representing the distribution of personalized features of users and items, we designed a sentiment-aware relationship weight and constructed two dictionaries, a dictionary of user history neighbors and weights, and a dictionary of item history neighbors and weights. Then, the proposed RAKCR is used to aggregate the neighborhood information perceived by the sentiment relationship weight in the knowledge graph to capture the personalized feature representations of both users and items, in order to better learn user and item embedding for more accurate personalized recommendations.

We apply our proposed RAKCR model in three different scenarios: movie, book, and business. The experiment results show that RAKCR achieves AUC gains of 2.1% to 10.3%, 2.4% to 11.5%, and 1.1% to 6.1% on the three datasets, respectively, compared to the state-of-the-art recommended baseline. In summary, the main contributions of this paper are as follows:

- A novel review sentiment-aware feature and knowledge graph alignment module is designed. By extracting the aspect sentiment dictionary through sentiment analysis of the reviews, the features in the sentiment dictionary are aligned with the corresponding nodes in the knowledge graph to construct a dictionary of adjacencies and weights for the sentiment perception of the reviews.
- A review sentiment-aware knowledge graph convolutional neural network is proposed. This is a generic recommendation framework that fuses reviews and knowledge graphs. With sentiment-aware relational weights, RAKCR is able to capture fine-grained personalized features of users and items, respectively.
- Experiments on three real datasets demonstrate the effectiveness of RAKCR on several state-of-the-art baselines.

2. Related work

2.1. Recommendation based on graph neural network

In recent years, graph neural networks have been increasingly used by researchers in recommendation algorithms due to their great success in graph applications (Wu et al., 2020; Wu, Sun, Zhang, Xie, & Cui, 2022). Graph neural networks iteratively aggregate neighborhood information by employing embedding propagation, which in turn mines the data for higher-order structural information. This approach can better represent the embedding of users or items in a recommendation system and greatly improve the performance of the recommendation algorithm (Gao et al., 2023; Ma, Guo, Li, & Zhao, 2022). For example, KGCN (Wang, Zhao, Xie, Li, & Guo, 2019) uses GCN to automatically capture high-order structural information and semantic information of items in the knowledge graph, and explore users' personalized preferences by learning the importance of relations to users. KGNN-LS (Wang, Zhang, Zhang et al., 2019) first converts the knowledge graph into a user-specific weighted graph, and then uses a graph neural network based on label smoothing regularization to compute personalized item embedding for recommender systems. KGAT (Wang, He, Cao, et al., 2019) uses user historical interaction data to model high-order relationships in a knowledge graph attention network to extract collaborative signals from collective behavior. MMGCN (Wei et al., 2019) uses GCNs to learn user- and micro-video-specific representations from multi-modal data to better capture user preferences for personalized micro-video recommendations. NIA-GCN (Sun et al., 2020) proposes a neighbor interaction-aware graph neural network framework, which significantly improves recommendation performance, in response to the problems of a discrepancy between user nodes and item nodes of the user–item bipartite graph ignored in the GCN recommendation algorithm and the inability to retain neighborhood relationship information in aggregation strategies. CGAT (Liu et al., 2021) utilizes a graph attention mechanism to extract local and non-local contextual information in knowledge graphs to better model user and item personalization features. LightGCN (He et al., 2020) simplifies the design of the GCN by retaining only the most important neighborhood aggregations, propagating the learning user and item embedding representations on the user–item interaction graph, and using a lightweight GCN better suited to the recommendation task. KLGCN (Wang, Li,

Zhang, & Wei, 2022) employs lightweight GCN to aggregate information from user–item interaction graphs and item KG, improving recommendation performance while reducing the number of model parameters. Huang et al. (2023) modeled the sequential patterns and temporal dynamics between user–item interactions through location-enhanced and time-aware graph convolutional networks, which in turn captured higher-order connections between users and items for predicting the next item in a session recommendation. Although these methods led to a significant improvement in recommendation performance, they ignored sentiment information between entities when aggregating neighborhood information using the GCN. Considering the sentiment information between entities when aggregated can better model personalized user and item representations, leading to more accurate personalized recommendations.

2.2. Review-based recommendations

Review data, as a naturally valid piece of auxiliary information, is often used in recommendation systems to improve interaction data sparsity and cold start problems (Sachdeva & McAuley, 2020). Compared to single rating data, review data can better mine the personalized interests of reviewers, thus enabling accurate personalized recommendations. A number of researchers have used review data to construct cross-domain recommendation strategies with a view to solving scientific problems in recommender systems (Fu, Peng, Wang, Xu, & Li, 2019; Zhao, Li, Xiao, Deng, & Sun, 2020). For example, RC-DFM (Fu et al., 2019) deeply fuses reviews and item content with rating matrices from both domains on the basis of a Stacked Denoising Autoencoder (SDAE), through which potential representations of users and items carrying more semantic information are learned. CATN (Zhao et al., 2020) designed an aspect transfer network for cross-domain recommendations, which first extracts multiple aspects of users and items from review texts and subsequently learns aspect relevance across domains using an attention mechanism, which aims to transfer user preferences from the secondary domain to the target domain to solve the user cold-start problem. Some researchers have used ratings and reviews to jointly capture potential user preferences for personalized recommendations. Xia et al. (2019) learn both rating and review features based on ratings and reviews, and use attention-based GRU networks to learn user and item information from user and item reviews. In some recent research, RGCL (Shuai et al., 2022) proposes a review-aware graph contrast learning framework. It first uses review data to construct user–item graphs augmented by edge features, with the aim of helping to learn the weights of each neighboring node at a fine-grained level for user and item representation learning. Secondly, two contrast learning tasks are designed to provide self-supervised signals for recommendations, thus improving recommendation performance. Park et al. (2022) constructed a sentiment-aware knowledge graph by analyzing users' ratings and reviews of items to add sentiment to the relationship, and used reinforcement learning strategies for item recommendation and reasoning.

The difference between the above work and ours is that the above work combines cross-domain knowledge to mine more user preference information from reviews, while our work focuses on mining the fine-grained sentiment characteristics of both users and items from reviews. The prior work constructs sentiment relationships by frequency statistics of sentiment polarity in reviews across all interaction histories. In contrast, our approach builds sentiment relationships on both the user and item sides. Furthermore, we model each interaction history separately at a fine-grained level, providing enhanced support for users' personalized preferences for individual items. Moreover, these approaches integrate sentiment features extracted from reviews with the existing knowledge graph, forming a novel knowledge graph. Subsequently, reinforcement learning strategies are employed for item reasoning based on this integrated knowledge graph. It is widely acknowledged that knowledge graphs consist of structured data, while user reviews are categorized as unstructured data. The fusion of these two disparate data types into a new knowledge graph is inevitably accompanied by the introduction of noise, directly influencing the performance of recommendation systems and, consequently, the accuracy of personalized recommendations. To enhance the specificity and effectiveness of personalized recommendations, this paper introduces sentiment features into the existing knowledge graph. This effectively reduces recommendation bias, achieving more accurate personalized recommendations.

2.3. Graph-based contrastive learning for recommendation

Recommendation methods based on graph contrastive learning have shown excellent performance in solving popularity bias and user–item interaction noise (Wei, Liang, Liu, & Wang, 2022; Xia, Huang, Xu, & Shi, 2023; Xia et al., 2022). SGL changes the graph structure in three ways: node dropout, edge dropout and random walk. Multiple views of nodes are subsequently generated for self-supervised learning on the user–item graph to enhance the accuracy and robustness of Graph Convolutional Networks (GCNs) in recommendation tasks (Wu et al., 2021). While MHCN encodes different types of higher-order user relations through multiple channels, obtaining comprehensive user representations through aggregation. Simultaneously, self-supervised learning is integrated into the hypergraph convolutional networks to cope with the possible loss problem of aggregation (Yu et al., 2021).CGI believes that the graph structures generated by these methods may not be optimal, prone to capturing information irrelevant to the recommendation task. Therefore, the CGI method constructs an optimized graph structure by adaptively removing nodes and edges to mitigate popularity bias. Additionally, by innovatively integrating the Information Bottleneck principle into the multi-view contrastive learning process, it can more effectively alleviate interaction noise (Wei et al., 2022).HCCF utilizes hypergraph structures to capture local and global collaborative relationships between users and items, combining it with contrastive learning to enhance the model's representational capabilities (Xia et al., 2022). On the other hand, KGCL designed a knowledge graph contrastive learning framework to guide data enhancement for user–item interaction graphs and mitigate noise issues (Yang et al., 2022). These methods employ heuristic-based contrast view generators, aiming to maximize mutual information between positive samples and have made good progress in separating negative instances. However, LightGCL argues that random perturbations may lead to the loss of useful structural information, and the heuristic-guided representation contrast scheme is limited by the view generator. Therefore, LightGCL simplifies the contrastive learning framework, proposing the use of singular value decomposition to guide graph augmentation, preserving crucial information about interactions (Cai, Huang, Xia, & Ren, 2023). AdaGCL creates adaptive contrast views through graph generation models and graph denoising models to address data noise and sparsity issues, thereby enhancing the robustness and performance of recommender systems within a self-supervised learning framework (Jiang, Huang, & Huang, 2023).

3. The proposed method

In this section, we introduce the proposed RAKCR model. We first formulate the reviews-aware recommendation problem. Then we detail the design of the proposed RAKCR model. Finally, we introduce the complete learning algorithm for the RAKCR model.

3.1. Problem formulation

In the recommendation system based on reviews and knowledge graph, let $U = \{u_1, u_2, \dots, u_M\}$ and $V = \{v_1, v_2, \dots, v_N\}$ denote M users and N items, respectively. The user–item interaction matrix $Y = \{y_{uv} | u \in U, v \in V\}$ is defined based on explicit feedback from users,

where the value of y_{uv} is 1 or 0. A value of 1 indicates an interaction between user u and item v, while 0 indicates no interaction. The set of reviews is denoted by $I = \{i_{uv}|u \in U, v \in V\}$, where i_{uv} represents the review text provided by user u for item v. Introduce the knowledge graph $\mathcal{G}$ as auxiliary information into the recommendation system, where $\mathcal{G}$ is a given knowledge graph relevant to the specific recommendation scenario. It contains a large number of entity–relation–entity triplets (h, r, t), where h and t are head and tail entities, belonging to ε, and r is the relation between entities, belonging to $\mathcal{R}$. In the knowledge-enhanced recommender system, the item v is associated with entities h or t in the given knowledge graph $\mathcal{G}$. By introducing triplets containing h or t, the knowledge graph can be integrated into the recommender system. The review i_{uv} contains fine-grained preferences of user u for item v. It is possible to extract the user's sentiment preferences for specific features of the item from the review. These specific features can be aligned with the corresponding relation in the given knowledge graph $\mathcal{G}$. Based on this alignment, the user's sentiment towards specific features from the reviews can be incorporated into the relation in the knowledge graph $\mathcal{G}$. For example, in the film knowledge graph, the triple (*Forrest Gump, film.film.actor, Tom Hanks*) indicates that Tom Hanks was an actor in Forrest Gump. Given the interaction matrix Y, the set of reviews I, and the knowledge graph $\mathcal{G}$, our aim is to predict the potential interest, i.e., the click probability, of user u in item v that has not been interacted with. Our goal is to learn a prediction function $\hat{y}_{uv} = \mathcal{F}(u, v|\Theta, Y, I, \mathcal{G})$, where $\hat{y}_{uv}$ denotes the probability of user u clicking on item v, and Θ denotes the model parameters of the function $\mathcal{F}$.

3.2. RAKCR layer

RAKCR is used to capture the fine-grained relational preferences of users and higher-order personalization features of items in the knowledge graph. It mainly consists of four modules: the review feature extraction module, the feature alignment module with entities in the knowledge graph, the neighbor entity and weight dictionary construction module, and the personalized feature aggregation module. The framework of RAKCR is shown in Fig. 1. The review feature extraction module uses sentiment analysis tools (referred to as SAT) to extract fine-grained sentiment preferences of users for items from reviews. Different colors in the figure represent different meanings, where gray represents users, green represents items, pink represents features in the reviews related to items, white represents sentiment, and blue represents other attributes of knowledge graph item nodes. The feature alignment module aims to associate the extracted sentiment from reviews with the relationships corresponding to the item nodes in the knowledge graph, where KG represents the given knowledge graph relevant to the specific recommendation scenario. The neighbor entity and weight dictionary construction module are created to aggregate personalized user and item embedding representations in KG, constructed with users as the center and with items as the center, respectively. In the personalized feature aggregation module, both the user side and the item side undergo multi-layer feature aggregation from multiple interaction histories, where l^* ($*$ takes values of 1, 2, 3) represents the number of graph convolution layers, and v^* or u^* represents multiple interaction histories (v^* represents multiple interaction history items for a user, and u^* represents multiple users interacting with a single item). RAKCR takes user u, item v, and user's reviews I of the item as input and outputs the predicted probability of user u clicking on item v. We will provide a detailed explanation of each module in the following content.

3.2.1. Review feature extraction module

As the data from users' reviews of the item contains fine-grained preferences for the item's characteristics, it is possible to extract aspects of the user's characteristics and corresponding sentiments about the item from the reviews. We used the methods in Lu, Castellanos, Dayal, and Zhai (2011), Zhang et al. (2014) for sentiment analysis of the reviews. The method takes reviews as input and outputs a sentiment lexicon $L = \{(f, o, s)|f \in F, o \in O, s \in S\}$, where F denotes a set of words associated with the item's characteristics, O denotes a set of words associated with the actual sentiment (e.g., nice, good, etc.), and S denotes a set of labels positive or negative for (f, o). For subsequent use, we processed the extracted sentiment dictionary into a quadruplet form as follows: $P = \{(u, v, f, s)|u \in U, v \in V, f \in F, s \in S\}$, where u denotes the user, v denotes the item, f denotes the feature associated with that item, and s denotes the sentiment polarity of user u with respect to feature f of item v, with 1 denoting positive sentiment and -1 denoting negative sentiment. For example, (Alice, Forrest Gump, actors, 1) means that the user Alice has given a positive sentiment rating to the actors of the movie Forrest Gump.

3.2.2. Feature alignment with entities in the knowledge graph

The sentiment-based aspect features we extract from user review texts include fine-grained sentiment representations of users on historical items, and fine-grained sentiment representations of multiple users on the same item. These sentiment feature pairs can be compared with the corresponding items in the knowledge graph. The specific relationship correspondence of the entity represents the user's different preferences for the specific relationship of the historical interaction items. For example, one user reviewed the movie Forrest Gump as "The costumes are mesmerizing and Tom Hanks' performance is unparalleled in its innocence and purity". From these, (costumes, positive) and (Tom Hanks, positive) can be extracted, which can be aligned to relations "film.film.costume_designer" and "film.film.star" respectively in the knowledge graph by means of the entity alignment algorithm. We employ Google's pre-trained word embedding model for alignment (Mikolov, Chen, Corrado, & Dean, 2013). Using the pre-trained model, we calculate the similarity between feature words and the neighboring relations of the corresponding item entities. Subsequently, we rank the obtained set of similarity scores from high to low, and the relation with the highest similarity value is the alignment target we desire. This relation represents the mapping of feature words from user reviews to relations in the knowledge graph. Through this mapping, we can associate the user's sentiment for specific features with the relations in the knowledge graph, indicating the user's preference for these relations. For example, in the given scenario, we need to align the feature word "costumes" from the review with the neighboring relations of the entity representing the movie "Forrest Gump" (such as costume_designer, star, genre, country, etc.). First, using the pre-trained model, calculate the similarity scores between "costumes" and "costume_designer, star, genre, country", assuming the scores are {0.7, 0.3, 0.2, 0.1}. After sorting, the highest similarity is found between "costumes" and "costume_designer", making the relation "costume_designer" the desired alignment target. The aligned lexicon of sentiment characteristics can be expressed in the following form: $\bar{P} = \{(u, v, r, s)|u \in U, v \in V, r \in \mathcal{R}, s \in S\}$, where r denotes the relationship with the highest similarity to the feature word in that item, i.e., user u holds emotion s for the relationship r of item v.

3.2.3. Dictionary of neighboring entities and weights

In order to better and more accurately represent user embeddings and item embeddings, more user features and item features need to be aggregated. There are rich semantic associations and feature information in the knowledge graph, which can well aggregate user and item representations. Aligning the sentiment features extracted from the reviews with their corresponding relationships in the knowledge graph enables a more accurate aggregation of the important information in the knowledge graph, which in turn filters the noise caused by unimportant features. By setting different weights for different sentiment perception relations, users' personalized preferences for historical items can be better aggregated, and user embeddings can be expressed more accurately. At the same time, for item embedding, it

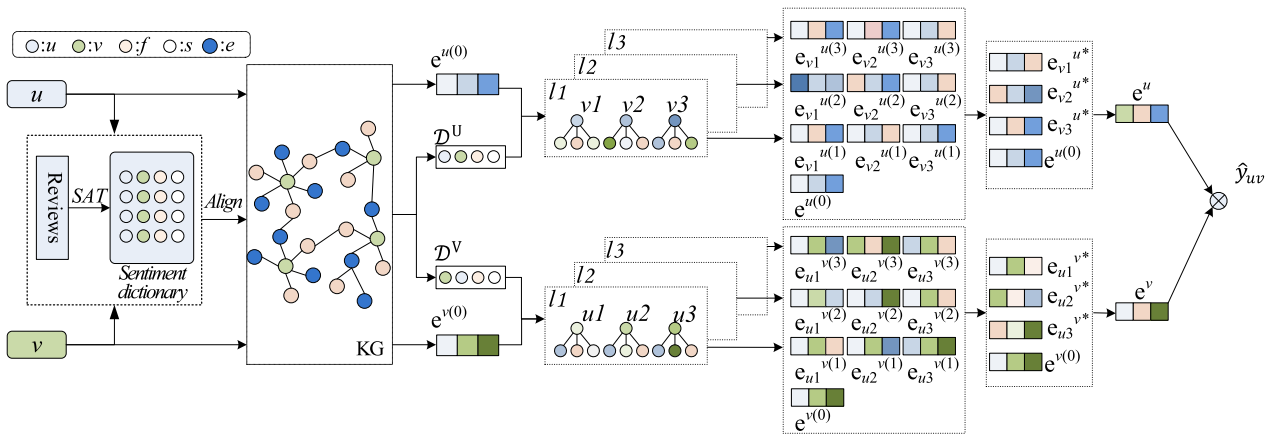

Fig. 1. The framework of RAKCR.

is possible to aggregate the fine-grained sentiment preferences of all users who have interacted with the item, based on which a mass-perceived representation of the item's characteristics can be formed, enabling a better representation of the item. As is well known, in a knowledge graph, different relations (edge labels) have varying degrees of importance in mining user preferences. Unlike traditional knowledge graph-enhanced recommendation algorithms, we align the edge labels with the fine-grained sentiments extracted from reviews to represent the user's sentiments towards different relations. To facilitate its integration into GCN, we further process the relations into the form of sentiment-aware relation weights. We represent the sentiment-aware relation weights as follows:

$$w_{r_{uv}} = \begin{cases} (1-\alpha) & , if\, s(u,v,r) = -1 \\ (2-\alpha) & , if\, s(u,v,r) = 1 \\ \alpha & , other \end{cases} \tag{1}$$

where s denotes the reviews sentiment polarity of user u for item entity v on relation r in the knowledge graph, 1 denotes positive sentiment, -1 denotes negative sentiment, and the other denotes no mention of the relation in the reviews, here denoted as neutral sentiment. The parameter α is a hyperparameter related to sentiment weight, with a range between 0.5 and 1. The parameter α represents the weight assigned to neutral sentiment and is specifically designed for aggregating personalized features. Typically, in knowledge graph-enhanced recommender systems, the weights of knowledge graph edges or relations are trained using a large amount of data, and the effectiveness of this training cannot be guaranteed, lacking interpretability. Here, the hyperparameter α and its variants $(1-\alpha)$ and $(2-\alpha)$, designed in this paper, are derived from the real sentiment expressions of users in the reviews. They accurately represent the weights of edges or relations. Based on these hyperparameters, personalized embedding representations aligning with user and item preferences can be aggregated, achieving truly personalized recommendations. We also constructed two dictionaries using user interaction history and knowledge graphs, a dictionary of user history neighbors and weights, and a dictionary of item history neighbors and weights, denoted as:

$$\begin{cases} \mathcal{D}^U = \{u : [h, t, w_{r_{uv}}]\} \\ \mathcal{D}^V = \{v : [h, t, w_{r_{uv}}]\} \end{cases} \tag{2}$$

$\mathcal{D}^U$ denotes a dictionary of neighbors and their corresponding relationship weights in the knowledge graph for historical interaction items of user u. $\mathcal{D}^V$ denotes a dictionary of neighbors and their corresponding relationship weights for all users who have interacted with item v on that item. Here, h represents the head entity in the knowledge graph, which can be either a item entity or a non-item entity, and t represents the tail entity connected to h. For example, in the triples $(ForrestGump, actor, TomHanks)$ and $(RobertZemeckis, directs, CastAway)$, h is $ForrestGump$ (project entity) and $RobertZemeckis$ (non-project entity), and t is $TomHanks$ and $CastAway$, respectively.$w_{r_{uv}}$ denotes the sentiment weight of user u on the relationship r between item entity v and neighboring entities. The role of $\mathcal{D}^U$ is to conveniently map the fine-grained sentiments of a user across multiple interaction histories to the edges or relations in the knowledge graph. When aggregating personalized embedding representations for users, the weight $w_{r_{uv}}$ is used to indicate the importance of the edge or relation. The role of $\mathcal{D}^V$ is to conveniently map the specific sentiments of multiple users who interacted with the item to the edges or relations in the knowledge graph. When aggregating personalized embedding representations for items, the weight $w_{r_{uv}}$ is used to indicate the importance of the edge or relation.

3.2.4. Personalized feature aggregation

We propose the RAKCR model to accurately capture the personalized characteristics of both users and items. Relational weights based on sentiment in terms of review text can better represent two aspects of information: first, the fine-grained personalized preference characteristics of users for historical interaction items, and second, the fine-grained personalized characteristics of items from a mass user perspective. The embedded representation of user u and item entity v candidate pairs can be more accurately represented by aggregating high-order fine-grained sentiment relationship weight-aware neighborhood information in knowledge graphs. First, we introduce the single-layer RAKCR. We obtain the interaction history neighborhood matrices $N^u(h,t)$ and $N^v(h,t)$ for u and v, and the corresponding weight matrices $w_{r_{uv}}$ from the user history neighborhood and weight dictionary $\mathcal{D}^U$ and the item history neighborhood and weight dictionary $\mathcal{D}^V$, respectively. It should be noted here that for user u, there may be multiple different history interaction items $[history_1, history_2, history_3, \ldots, history_k]$. User u has different personalized interests on different items, so the relationship weights on different items are different. For item v, on the other hand, there may be multiple users who have interacted with the item before. Different users have different concerns about the item and therefore have different relationship weights on the item. The item history neighborhood and weight matrix represent the neighbors of different users on item v and the corresponding sentiment weights. The normalized relational weight matrix can be expressed as:

$$\widetilde{w}_{r_{uv}} = \frac{exp(w_{r_{uv}})}{\sum_{r \in R} exp(w_{r_{uv}})} \tag{3}$$

Where $\widetilde{w}_{r_{uv}}$ denotes the weight of the normalized sentiment-perception relationship between user u's historical interaction item entity v and its neighbor nodes, which is our carefully designed weight parameter for the importance of using reviews of sentiment-perception relationships. When forward propagation takes place, it focuses on information that

is of interest to the user. Different from the traditional GCN, we extract the user's personalized interest features and item features from the public user's perspective from the reviews, and specify the importance of different neighbors instead of getting the importance of neighbors through training. The linear combination of neighbors of the interaction history can thus be expressed as:

$$e_{N_h} = \sum_{h,t \in N(h,t)} \widetilde{w}_{r_{uv}} e_t \tag{4}$$

Where e_t denotes the embedding representation of the tail entity t connected to the neighboring head entity h. The representation e_h of the head entity h is then aggregated with the neighboring representation e_{N_h} into a new representation $e_h^* = f_{agg}(e_h, e_{N_h})$ of the head entity h. We use three types of $f_{agg}(\cdot)$ aggregators in RAKCR:

• **Sum Aggregator**(Kipf & Welling, 2016) sums the item entity representation with its neighbor representations, followed by a linear transformation.

$$e_h^* = LeakyReLU(W_1 \cdot (e_h + e_{N_h}) + b) \tag{5}$$

• **Concat Aggregator** (Hamilton, Ying, & Leskovec, 2017) concatenates two embedded representations and performs a linear transformation.

$$e_h^* = LeakyReLU(W_1 \cdot concat(e_h + e_{N_h}) + b) \tag{6}$$

• **Neighbor Aggregator** (Velickovic et al., 2017) only uses the neighborhood representation as the output representation of this layer.

$$e_h^* = LeakyReLU(W_1 \cdot e_{N_h} + b) \tag{7}$$

e_h^* in the above formula is the aggregation result obtained by the single-layer GCN. The representation of an item and its neighbors are bound together by an aggregation operation, and the fine-grained sentiment features extracted from reviews are fused into personalized representations of users and items. To obtain more information, we propagate higher-order neighborhood information and represent the entity aggregation at layer l as:

$$e_h{}^{(l)} = f_{agg}(e_h{}^{(l-1)}, e_{N_h}{}^{(l-1)}) \tag{8}$$

$e_h{}^{(l)}$ and $e_{N_h}{}^{(l-1)}$ denote the head entity and neighbor representation of the previous layer, respectively, and l is a hyperparameter indicating the number of layers of GCN. Therefore, the final embedding of a history for a user or item can be expressed as:

$$e_{history} = e_h^{(0)} \| \ldots \| e_h^{(l)} \tag{9}$$

For multiple historical interactions, different weights need to be assigned as different users have different preferences. A weight W_2 is set here to indicate the distribution of weights between different histories, and W_2 is a trainable parameter. The above formulas all apply to the embedding representation of user u and item v. The embedding of end users and items can be expressed as:

$$\begin{cases} e^u = \sigma\left(\sum_{i=1}^{k} W_2^i \cdot e_{history}^i + e^{u(0)}\right) \\ e^v = \sigma\left(\sum_{i=1}^{k} W_2^i \cdot e_{history}^i + e^{v(0)}\right) \end{cases} \tag{10}$$

Where $e_{history}^i$ is the embedded representation of the ith interaction history of user u or item v, i takes values from 1 to k, and k is a hyperparameter indicating the number of interaction histories. W_2^i is the weight of the ith interaction history, and $e^{u(0)}$ and $e^{v(0)}$ denote the initially embedded representations of user u and item v, respectively. σ is a non-linear function such as ReLU.

According to the embedded representation e^u of user u and the embedded representation e^v of item v, the probability of user u clicking on item v can be predicted by using the function $\mathcal{F} : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$:

$$\hat{y}_{uv} = \mathcal{F}(e^u, e^v) \tag{11}$$

3.3. Learning algorithm

In our RAKCR model, we first get the historical neighbors and weights of user u and item v. Each user and item has multiple interaction histories, and multiple interaction histories are performed simultaneously during aggregation without interfering with each other. New representations of multiple interaction histories (depending on self and neighborhood representations) can be obtained through single-layer RAKCR aggregation. Extending RAKCR from one layer to multiple layers for mining higher-order personalized features of users and items. Through multi-layer propagation, the aggregate representation of users and items in multiple interaction histories is obtained. Note here that users or items have a representation of each interaction history through multiple layers of aggregation, indicating personalized preferences for that interaction history. Finally, through an MLP layer, different weights are assigned to different interaction histories to obtain the final embedding representations of users and items, which represent different degrees of preference for each interaction history. We give a formal description of the above steps in Algorithm 1.k denotes the number of interaction histories and l denotes the number of layers in the model.

Algorithm 1 RAKCR algorithm

Input: Interaction matrix Y; Reviews set I; Knowledge graph $\mathcal{G}(\mathcal{E}, \mathcal{R})$; Model training parameters Θ; Number of interaction histories k; Number of layers in the model l
Output: Prediction function $\mathcal{F}(u, v \mid \Theta, Y, I, \mathcal{G})$
  Initialize all parameters;
  Constructing a dictionary of users' history neighbors and weights D^U, a dictionary of items' history neighbors and weights D^V;
  for $(u, v) \in Y$ **do**
    Get historical neighbors and weights for users and items;
    for $k = 1, \ldots, k$ **do**
      for $l = 1, \ldots, l$ **do**
        $e_{N_h} \leftarrow$ Linear combination neighbors;
        $e_h{}^{(l)} \leftarrow$Aggregating neighbors and entities h;
      end for
      $e_{history} \leftarrow$Multi-layer stitching to obtain an interactive historical representation;
    end for
    $e^u, e^v \leftarrow$ Combine the representation of all interaction history of a user or project with the initial user and project representation;
    Calculating the predicted probability
    $\hat{y}_{uv} = \mathcal{F}(e^u, e^v)$;
    Update parameters by gradient descent;
  end for
  return $\mathcal{F}(u, v \mid \Theta, Y, I, \mathcal{G})$

To optimize our model, we use Bayesian Personalized Ranking (BPR) loss (Rendle, Freudenthaler, Gantner, & Schmidt-Thieme, 2012). This loss function training model predicts the observed items higher than the unobserved items:

$$\mathcal{L} = -\sum_{u,i,j \in O} ln\sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|\mathcal{F}\|_2^2 \tag{12}$$

Where $O = \{(u,i,j) | (u,i) \in Y, y_{ui} = 1; (u,j) \in Y, y_{uj} = 0\}, \sigma(\cdot)$ is the activation function and the last term is the $L2$ regularization term to prevent overfitting of the model.

4. Experiments

In this section, we evaluate the proposed RAKCR model on three real-world scenario datasets: film, book, and business, respectively.

Table 1
Statistics for the three datasets.

	MovieLens-20M	Amazon-Book	Yelp
users	316,258	839,644	117,370
items	16,954	24,915	150,346
interactions	1,421,777	1,902,124	4,392,169
entities	102,569	88,572	155,466
relations	32	39	42
KG triples	499,474	2,557,746	1,566,773

4.1. Data sets

To evaluate the performance of RAKCR, we use the following datasets for three different recommendation scenarios: MovieLens-20M, Amazon-book, and Yelp, described in detail as follows:

• MovieLens-20M[1]: Our work requires a dataset with review data corresponding to ratings, and the MovieLens dataset, which is commonly used in recommendation algorithms, does not officially have review data available. Therefore, on the basis of [6], we crawled and built a new dataset with review data based on the items through IMDB, the data source website of MovieLens. To ensure the quality of the dataset, we use a 10-core setting, which guarantees at least 10 interacting users and items.

• Amazon-book[2]: Amazon-book is a commonly used dataset in knowledge graph recommendation algorithms (He & McAuley, 2016; McAuley, Targett, Shi, & Van Den Hengel, 2015). We downloaded the dataset with the reviews data, and we also used the 10-core setting to ensure the quality of the dataset.

• Yelp[3]: The Yelp dataset consists of data from approximately 150,000 merchants, 6.99 million reviews, and 200,000 images from 11 metropolitan areas. We use restaurants, bars, and other merchants as items. Again using the 10-core setting.

We use user rating data as well as review data in our work. In addition to the rating and review data from user–item interactions, we configure the knowledge graph data for each dataset. For MovieLens-20M, we use the knowledge graph provided in Wang, Zhao, et al. (2019). For Amazon-book, we used the knowledge graph provided in Wang, He, Cao, et al. (2019). For Yelp, we extracted item knowledge from the local business information network as knowledge graph data. The data statistics for the three datasets are shown in Table 1.

4.2. Baselines

To validate the effectiveness of our proposed RAKCR model, we compared it with several types of recommendation algorithms: collaborative filtering-based approaches (BPRMF), embedding-based approaches (CKE, CFKG), knowledge graph-based approaches (RipplNet, KGCN, KG-AT,KGCL), and GNN-based methods (KGCN, KGAT, LightGCN). These are shown below:

• BPRMF (Rendle et al., 2012): This is a typical CF-based approach that uses paired matrix decomposition for personalized ranking algorithm optimization on implicit feedback data.

• CFKG (Ai, Azizi, Chen, & Zhang, 2018): This method uses a knowledge base representation learning approach to learn heterogeneous entity embedding representations and provide a personalized interpretation of the recommended items.

•RippleNet (Wang et al., 2018): This is a representative approach to knowledge graph recommendation, which uses knowledge graphs to iteratively propagate user preferences, extend potential user interests and personalize recommendations.

•KGCN (Wang, Zhao, et al., 2019): This is the classic knowledge graph and GNN-based approach. It uses GCN to automatically capture higher-order structural and semantic information about items in the knowledge graph and to explore the personalized preferences of users by learning the importance of relationships with them.

• KGAT (Wang, He, Cao, et al., 2019): This method uses historical user interaction data to model higher-order relationships in a knowledge graph attention network, thereby extracting collaborative signals from collective behavior for use in recommendations.

• LightGCN (He et al., 2020): This approach simplifies the design of the GCN by keeping only the most important neighborhood aggregations, propagating the learning user and item embedding representations on the user–item interaction graph, and using the lightweight GCN for recommendation tasks.

• KGCL (Yang et al., 2022): This method effectively utilizes information from the knowledge graph by combining it with contrastive learning, addressing noise issues in the knowledge graph, and enhancing the accuracy and robustness of the recommender system.

4.3. Experimental setup

We configured our experimental environment using Python 3.7, Tensorflow 1.14.0, and numpy 1.18.0, on which we implemented our RAKCR model. We conduct experimental validation under two types of scenarios: (1) In the click-through rate prediction scenario, we use AUC and F1 to evaluate the prediction results; (2) In the top-K recommendation scenario, we select the k items with the highest CTR prediction rankings for each user on the test set, and use recall@k to evaluate the recommendation results. We divided each dataset into a training set, a validation set, and a test set, with the ratio set at 6:2:2. In our RAKCR model, α is a hyperparameter, taking values between 0.5 and 1, indicating the sentiment perception weight parameter, which is a parameter we have specifically designed for aggregating personalized features. The parameter experiments in Table 6 indicate that the performance is optimal when α is set to 0.7. We set the f function to the inner product, the last layer aggregator to tanh, and the non-last layer to use LeakyReLU as the activation function. We use the Adam optimizer for model optimization. Table 1 lists the detailed settings of other hyperparameters on the three datasets, respectively.

We used grid search to find the optimal hyperparameter settings. The learning rate was adjusted between $\{5e-2, 2e-2, 5e-3, 2e-3\}$, the $L2$ normalization factor was adjusted between $\{1e-4, 1e-5, 1e-6, 1e-7\}$, the adoption neighbor size was adjusted between $\{4, 6, 8, 10\}$, the embedding dimension was adjusted between $\{8, 16, 32, 64\}$, and the sentiment perception weights were adjusted between $\{0.5, 0.6, 0.7, 0.8\}$.

The hyperparameter settings for the baseline models are as follows. For RippleNet, the embedding dimensions (d) and learning rates (η) for the three datasets are: for MovieLens-20M and Amazon-book, $d = 16$, $\eta = 0.005$; for Yelp, $d = 32$, $\eta = 0.002$. The hop number (H) is set to 3. For KGCN, the dimensions and learning rates are 16,32,32 and 0.005,0.002,0.02 respectively, with a batch size of 65536. For KGAT, the embedding dimension is fixed at 64, batch size at 1024, the number of layers (l) is set to 3, and hidden dimensions are 64,32,16. For LightGCN, the values of the $L2$ regularization coefficient λ for the three datasets are 0.0001, 0.0001, and 0.001, with a learning rate of 0.001. For KGCL, the embedding dimension is fixed at 64, learning rate is 0.001, and the contrastive regularization parameter λ_1 is set to 0.1, with a hard negative sampling parameter τ of 0.2. Other hyperparameter settings for the baseline models follow those specified in the original papers.

4.4. Results

The experimental results of the proposed RAKCR model in both top-K and CTR prediction scenarios are presented in Table 2 and Fig. 2,

[1] https://grouplens.org/datasets/movielens/
[2] http://jmcauley.ucsd.edu/data/amazon
[3] https://www.yelp.com/dataset

Table 2
The results of AUC and F1 in CTR prediction.

Model	MovieLens-20M		Amazon-book		Yelp	
	AUC	*F1*	*AUC*	*F1*	*AUC*	*F1*
RippleNet	0.865(+8.3%)	0.792(+9.3%)	0.886(+6.3%)	0.825(+7.9%)	0.894(+5.6%)	0.872(+7.3%)
KGCN	0.870(+7.7%)	0.819(+6.2%)	0.903(+4.5%)	0.844(+5.8%)	0.916(+3.3%)	0.909(+3.4%)
CKE	0.871(+7.6%)	0.785(+10.1%)	0.839(+11.3%)	0.812(+9.4%)	0.915(+3.4%)	0.887(+5.7%)
CFKG	0.853(+9.5%)	0.781(+10.5%)	0.884(+6.6%)	0.836(+6.7%)	0.889(+6.1%)	0.879(+6.6%)
BPRMF	0.846(+10.3%)	0.763(+12.6%)	0.837(+11.5%)	0.807(+9.9%)	0.909(+4.0%)	0.898(+4.6%)
KGAT	0.915(+3.0%)	0.839(+3.9%)	0.923(+2.4%)	0.875(+2.3%)	0.925(+2.3%)	0.922(+2.0%)
LightGCN	0.923(+2.1%)	0.854(+2.2%)	0.921(+2.6%)	0.863(+3.7%)	0.937(+1.1%)	0.933(+0.9%)
KGCL	0.916(+2.9%)	0.852(+2.4%)	0.919(+2.8%)	0.859(+4.1%)	0.935(+1.3%)	0.913(+3.0%)
OURS	**0.943**	**0.873**	**0.946**	**0.896**	**0.947**	**0.941**

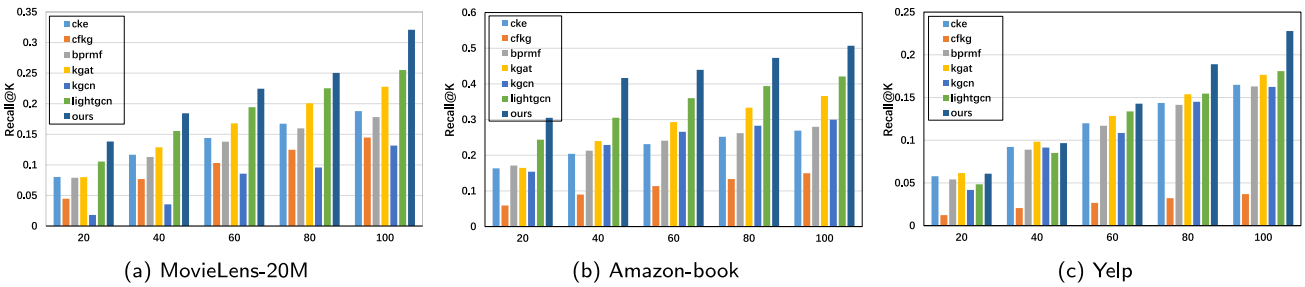

Fig. 2. The results of Recall@K in top-K recommendation.

respectively. In Table 2, the symbol "+" indicates an improvement of the RAKCR model compared to the baseline model. Observations from Table 2 and Fig. 2 include:

• Overall, RAKCR outperformed the baseline model on all three datasets, indicating that the sentiment features extracted from the reviews helped to improve the accuracy of personalized recommendations.

• Models such as Ripplenet, KGCN, KGCL, and KGAT that use knowledge graphs outperform models such as CKE and BPRMF that do not use knowledge graphs, which shows that knowledge graphs play an important role in improving recommendation performance as auxiliary information.

• The KGCN, KGAT, and LightGCN models using GCN are significantly better than the others, which shows the advantages of using GCN for recommendation algorithms.

• Overall, our proposed model achieves higher performance gains on the movie and book datasets than the yelp dataset. This phenomenon suggests that RAKCR can alleviate the data sparsity problem, as the MovieLens-20M and Amazon-book datasets are sparser than the yelp dataset.

• Among all baseline models, LightGCN performs the best because it retains the most important neighborhood aggregation, indicating that aggregating information from the neighborhood is crucial for recommendation algorithms.

• Compared to knowledge graph-based recommendation methods such as RippleNet, KGCN, and KGAT, the overall performance of KGCL, which also utilizes the knowledge graph, consistently outperforms these methods. Among these approaches, only KGCL employs contrastive learning to enhance the knowledge graph structure and address knowledge graph noise issues. This indicates the effectiveness of using contrastive learning for graph enhancement strategies.

4.4.1. The impact of different aggregators

The comparison results of using different aggregators in our proposed RAKCR model are shown in Table 3. Based on the results in the table, we can draw the following conclusions:

• Among the three aggregators, the sum aggregator has the best results, which may be due to the fact that the concat aggregator introduces noise by merging irrelevant information into the embedding representation, while the neighbor aggregator discards important information about the head entity itself in its embedding representation.

Table 3
Effects of different aggregators.

Aggregator	MovieLens-20M		Amazon-book		Yelp	
	AUC	*F1*	*AUC*	*F1*	*AUC*	*F1*
Sum	**0.943**	**0.873**	**0.946**	**0.896**	**0.947**	**0.941**
Concat	0.941	0.870	0.944	0.895	0.946	0.941
Neighbor	0.933	0.862	0.935	0.887	0.937	0.936

Table 4
Effects of different layers.

	MovieLens-20M		Amazon-book		Yelp	
	AUC	*F1*	*AUC*	*F1*	*AUC*	*F1*
RAKCR-1	0.942	0.869	0.942	0.891	0.946	0.937
RAKCR-2	0.943	0.873	0.946	0.896	**0.947**	0.941
RAKCR-3	**0.943**	**0.873**	**0.946**	**0.898**	0.946	**0.941**
RAKCR-4	0.926	0.852	0.935	0.874	0.932	0.928

• The results of the Neighbor aggregator are not significantly lower than those of Sum aggregation and concat aggregation, suggesting that aggregating personalized information from neighbors is crucial for embedding representations.

4.4.2. The impact of different numbers of layers

We studied the impact of different numbers of layers in RAKCR, and the comparison results are shown in Table 4. Based on the results in the table, we can draw the following conclusions:

• Overall, the performance of the model gradually improves as the number of layers increases, indicating that our model can effectively capture higher-order relationships to obtain personalized user and item embedding representations.

• When the number of embedding layers in the model increases to 4, we observe a significant decrease in performance, which indicates that when the number of layers increases to a certain extent, the extracted user and item features decrease, and noise is introduced, leading to a decline in model performance.

4.4.3. The impact of different numbers of sampled neighbors

We studied the impact of the number of sampled neighbors on model results by changing the number of neighbors used in each layer

Table 5
Effect of different numbers of sampled neighbors.

	MovieLens-20M		Amazon-book		Yelp	
	AUC	*F1*	*AUC*	*F1*	*AUC*	*F1*
4	0.943	0.873	0.946	0.896	0.947	0.941
6	**0.947**	0.877	**0.948**	**0.899**	**0.950**	**0.942**
8	0.945	**0.878**	0.945	0.895	0.948	0.941
10	0.874	0.836	0.883	0.841	0.892	0.886

Table 6
The impact of different alpha values on the performance of RAKCR.

Alpha	MovieLens-20M		Amazon-book		Yelp	
	AUC	*F1*	*AUC*	*F1*	*AUC*	*F1*
0.6	0.939	0.866	0.938	0.882	0.942	0.939
0.7	**0.943**	**0.873**	**0.946**	**0.896**	**0.947**	**0.941**
0.8	0.941	0.870	0.938	0.889	0.945	0.940
0.9	0.937	0.867	0.931	0.887	0.944	0.938

of GCN. The comparison results using different numbers of sampled neighbors are shown in Table 5. We can observe from the table that RAKCR performs best when the sampling size is 6, and its performance significantly decreases when the sampling size is 10. This indicates that increasing the sampling size appropriately can improve performance, but too many sampled neighbors can introduce noise and negatively impact performance.

4.4.4. The impact of different alpha values on the performance of RAKCR

To explore the impact of the sentiment weight hyperparameter alpha on the model, in this section, we investigate the performance of RAKCR in the CTR prediction recommendation scenario with different values of alpha. Since alpha ranges from 0.5 to 1, we tested RAKCR performance with alpha values of 0.6, 0.7, 0.8, and 0.9. The experimental results are shown in Table 6. From the results, it is clear that within the tested range, as the alpha value increases, the performance of RAKCR first rises and then falls. When alpha is set to 0.7, the model achieves optimal performance. This indicates that an appropriate alpha value is helpful for improving the model's performance. Additionally, another interesting observation is that the performance of RAKCR does not fluctuate significantly with different alpha values. Moreover, the performance consistently surpasses the baseline model, indicating that the specially designed sentiment weight hyperparameter is essential. Using this unique weight to aggregate information in the knowledge graph allows for a more accurate propagation of user personalized preferences, filtering out noise data irrelevant to user preferences.

4.4.5. The impact of the sentiment component on the model

To investigate the impact of the sentiment component on the performance of the RAKCR model, we designed a variant of the model, RAKCR w/o Sentiment, which excludes the sentiment component and only uses the initial knowledge graph for testing RAKCR's performance. The results of the variant compared to the complete RAKCR are shown in Table 7. From the results in Table 7, it is evident that the performance of the RAKCR model consistently outperforms the RAKCR variant on the MovieLens-20M, Amazon-Book, and Yelp datasets. This indicates that the proposed sentiment-aware component in reviews effectively enhances the performance of knowledge graph-enhanced recommendations.

4.5. Discussion

In order to more intuitively and accurately capture users' personalized preferences and improve the accuracy of personalized recommendation algorithms, this paper proposes the RAKCR model. The proposed RAKCR model was trained and validated on three datasets: MovieLens-20M, Amazon-Book, and Yelp. The model was evaluated

Table 7
The impact of the sentiment component on the model.

Variant	MovieLens-20M		Amazon-book		Yelp	
	AUC	*F1*	*AUC*	*F1*	*AUC*	*F1*
RAKCR w/o Sentiment	0.878	0.813	0.907	0.835	0.923	0.918
RAKCR	**0.943**	**0.873**	**0.946**	**0.896**	**0.947**	**0.941**

in two recommendation scenarios: CTR prediction and top-K prediction. Generally, the proposed RAKCR model achieved substantial improvements across all three datasets. Compared to the classical collaborative filtering algorithm BPRMF, in the CTR prediction scenario, the AUC and F1 metrics improved by 10.3% and 12.6%, respectively, for the MovieLens-20M dataset; for the Amazon-Book dataset, the improvements were 11.5% and 9.9%, and for the Yelp dataset, the improvements were 4.0% and 4.6%. In comparison with embedding-based methods like CKE and CFKG, in the CTR prediction scenario, the AUC and F1 metrics improved by an average of 8.5% and 7.8%, respectively, for the MovieLens-20M dataset; for the Amazon-Book dataset, the improvements were 9.0% and 8.2%, and for the Yelp dataset, the improvements were 4.8% and 6.1%. Compared to knowledge graph-based methods such as RippleNet, KGCN, KGAT, and KGCL, in the CTR prediction scenario, the AUC and F1 metrics improved by an average of 5.5% and 5.4%, respectively, for the MovieLens-20M dataset; for the Amazon-Book dataset, the improvements were 4.0% and 5.0%, and for the Yelp dataset, the improvements were 3.1% and 3.9%. Compared to graph neural network-based methods (KGCN, KGAT, and LIGHTGCN), in the CTR prediction scenario, the AUC and F1 metrics improved by an average of 4.3% and 4.1%, respectively, for the MovieLens-20M dataset; for the Amazon-Book dataset, the improvements were 3.2% and 3.9%, and for the Yelp dataset, the improvements were 2.2% and 2.1%. In the top-K recommendation scenario, the experimental results on the three datasets show that RAKCR performs significantly well, especially in the top-80/100 metrics.

From the above analysis, it is evident that overall, RAKCR exhibits the most significant improvement compared to collaborative filtering methods. This suggests that RAKCR is more effective than other methods in mitigating the data sparsity and cold start issues associated with collaborative filtering. From a dataset perspective, RAKCR's average performance improvement on the MovieLens-20M and Amazon-Book datasets is notably higher than on the Yelp dataset, which is relatively denser. This further emphasizes RAKCR's contribution to alleviating data sparsity issues. The performance of RAKCR in top-K recommendations indicates a significant improvement in recommendation accuracy compared to baseline models, especially in scenarios with data sparsity (MovieLens-20M and Amazon-Book datasets), where the accuracy improvement is more pronounced. This phenomenon further validates the importance of emotion-aware relation weights in exploring personalized features of users and items. The discussion of the results above suggests that the personalized recommendation achieved by capturing fine-grained sentiment features from reviews successfully integrates emotional elements into external auxiliary information. Although the proposed RAKCR model demonstrates good performance, it is limited to incorporating sentiment features from the textual modality. Other modalities' sentiment features can also serve as essential information sources for mining personalized user representations.

5. Conclusion

In this paper, we explore the role of review feature sentiment in knowledge graph personalized feature extraction for personalized recommendations. We propose a unified framework, RAKCR, to provide more accurate personalized recommendations by mining fine-grained personalized features in reviews. The approach aligns sentiment features extracted from reviews in a knowledge graph, aiming to iteratively aggregate personalized feature representations of users and

items via GCN. Extensive experiments on three real-world datasets demonstrate the effectiveness of RAKCR. For future work, we plan to (1) further explore ways to align and fuse review features with the knowledge graph. (2) In this paper, we used custom-weighted GCN to aggregate neighborhood information in the knowledge graph. Another focus of future work is how to better use GCN to aggregate personalized preferences of users and items.(3) Further explore fine-grained user sentiments in multimodal information, extract user preferences from multimodal interaction data, and use them for personalized and diverse recommendations. Simultaneously, we plan to apply the proposed model in the field of education.

CRediT authorship contribution statement

Yachao Cui: Conceptualization, Methodology, Software, Writing – original draft, Funding acquisition, Data curation, Investigation. **Hongli Yu:** Investigation, Software, Methodology. **Xiaoxu Guo:** Investigation, Methodology. **Han Cao:** Supervision, Funding acquisition. **Lei Wang:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the Fundamental Research Funds for the Central Universities (No. GK202303006).

References

Ai, Q., Azizi, V., Chen, X., & Zhang, Y. (2018). Learning heterogeneous knowledge base embeddings for explainable recommendation. *Algorithms*, *11*(9), 137.

Cai, X., Huang, C., Xia, L., & Ren, X. (2023). LightGCL: Simple Yet effective graph contrastive learning for recommendation. In *The eleventh international conference on learning representations*.

Chen, T., He, X., & Kan, M.-Y. (2016). Context-aware image tweet modelling and recommendation. In *Proceedings of the 24th ACM international conference on multimedia* (pp. 101–1027).

Cheng, Z., Chang, X., Zhu, L., Kanjirathinkal, R. C., & Kankanhalli, M. (2019). MMALFM: Explainable recommendation by leveraging reviews and images. *ACM Transactions on Information Systems (TOIS)*, *37*(2), 1–28.

Fu, W., Peng, Z., Wang, S., Xu, Y., & Li, J. (2019). Deeply fusing reviews and contents for cold start users in cross-domain recommendation systems. *Vol. 33*, In *Proceedings of the AAAI conference on artificial intelligence* (01), (pp. 94–101).

Gao, C., Zheng, Y., Li, N., Li, Y., Qin, Y., Piao, J., et al. (2023). A survey of graph neural networks for recommender systems: Challenges, methods, and directions. *ACM Transactions on Recommender Systems*, *1*(1), 1–51.

Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. In *Advances in neural information processing systems*: *vol. 30*.

He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). Lightgcn: Simplifying and powering graph convolution network for recommendation. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (pp. 639–648).

He, R., & McAuley, J. (2016). Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In *Proceedings of the 25th international conference on world wide web* (pp. 507–517).

Huang, L., Ma, Y., Liu, Y., Danny Du, B., Wang, S., & Li, D. (2023). Position-enhanced and time-aware graph convolutional network for sequential recommendations. *ACM Transactions on Information Systems*, *41*(1), 1–32.

Jamali, M., & Ester, M. (2010). A matrix factorization technique with trust propagation for recommendation in social networks. In *Proceedings of the fourth ACM conference on recommender systems* (pp. 135–142).

Jiang, Y., Huang, C., & Huang, L. (2023). Adaptive graph contrastive learning for recommendation. In *Proceedings of the 29th ACM SIGKDD conference on knowledge discovery and data mining* (pp. 4252–4261).

Kipf, T. N., & Welling, M. (2016). Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.

Koren, Y. (2008). Factorization meets the neighborhood: a multifaceted collaborative filtering model. In *Proceedings of the 14th ACM SIGKDD international conference on knowledge discovery and data mining* (pp. 426–434).

Liu, Y., Miyazaki, J., & Chang, Q. (2022). Knowledge graph enhanced multi-task learning between reviews and ratings for movie recommendation. In *Proceedings of the 37th ACM/SIGAPP symposium on applied computing* (pp. 1882–1889).

Liu, Y., Yang, S., Xu, Y., Miao, C., Wu, M., & Zhang, J. (2021). Contextualized graph attention network for recommendation with item knowledge graph. *IEEE Transactions on Knowledge and Data Engineering*, *35*(1), 181–195.

Lu, Y., Castellanos, M., Dayal, U., & Zhai, C. (2011). Automatic construction of a context-aware sentiment lexicon: an optimization approach. In *Proceedings of the 20th international conference on world wide web* (pp. 347–356).

Ma, R., Guo, F., Li, Z., & Zhao, L. (2022). Knowledge graph random neural networks for recommender systems. *Expert Systems with Applications*, *201*, 117–120.

McAuley, J., Targett, C., Shi, Q., & Van Den Hengel, A. (2015). Image-based recommendations on styles and substitutes. In *Proceedings of the 38th international ACM SIGIR conference on research and development in information retrieval* (pp. 43–52).

Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. arXiv preprint arXiv:1301.3781.

Park, S.-J., Chae, D.-K., Bae, H.-K., Park, S., & Kim, S.-W. (2022). Reinforcement learning over sentiment-augmented knowledge graphs towards accurate and explainable recommendation. In *Proceedings of the fifteenth ACM international conference on web search and data mining* (pp. 784–793).

Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2012). BPR: Bayesian personalized ranking from implicit feedback. arXiv preprint arXiv:1205.2618.

Sachdeva, N., & McAuley, J. (2020). How useful are reviews for recommendation? a critical review and potential improvements. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (pp. 1845–1848).

Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th international conference on world wide web* (pp. 285–295).

Shuai, J., Zhang, K., Wu, L., Sun, P., Hong, R., Wang, M., et al. (2022). A review-aware graph contrastive learning framework for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 1283–1293).

Sun, J., Zhang, Y., Guo, W., Guo, H., Tang, R., He, X., et al. (2020). Neighbor interaction aware graph convolution networks for recommendation. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (pp. 1289–1298).

Tang, X., Wang, T., Yang, H., & Song, H. (2019). AKUPM: Attention-enhanced knowledge-aware user preference model for recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 1891–1899).

Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al. (2017). Graph attention networks. *Stat*, *1050*(20), 10–48550.

Wang, X., He, X., Cao, Y., Liu, M., & Chua, T.-S. (2019). Kgat: Knowledge graph attention network for recommendation. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 950–958).

Wang, X., He, X., Wang, M., Feng, F., & Chua, T.-S. (2019). Neural graph collaborative filtering. In *Proceedings of the 42nd international ACM SIGIR conference on research and development in information retrieval* (pp. 165–174).

Wang, F., Li, Y., Zhang, Y., & Wei, D. (2022). KLGCN: Knowledge graph-aware Light Graph Convolutional Network for recommender systems. *Expert Systems with Applications*, *195*, Article 116513.

Wang, H., Zhang, F., Wang, J., Zhao, M., Li, W., Xie, X., et al. (2018). Ripplenet: Propagating user preferences on the knowledge graph for recommender systems. In *Proceedings of the 27th ACM international conference on information and knowledge management* (pp. 417–426).

Wang, H., Zhang, F., Zhang, M., Leskovec, J., Zhao, M., Li, W., et al. (2019). Knowledge-aware graph neural networks with label smoothness regularization for recommender systems. In *Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining* (pp. 968–977).

Wang, H., Zhao, M., Xie, X., Li, W., & Guo, M. (2019). Knowledge graph convolutional networks for recommender systems. In *The world wide web conference* (pp. 3307–3313).

Wei, C., Liang, J., Liu, D., & Wang, F. (2022). Contrastive graph structure learning via information bottleneck for recommendation. *Advances in Neural Information Processing Systems*, *35*, 20407–20420.

Wei, Y., Wang, X., Nie, L., He, X., Hong, R., & Chua, T.-S. (2019). MMGCN: Multi-modal graph convolution network for personalized recommendation of micro-video. In *Proceedings of the 27th ACM international conference on multimedia* (pp. 1437–1445).

Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., & Philip, S. Y. (2020). A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(1), 4–24.

Wu, S., Sun, F., Zhang, W., Xie, X., & Cui, B. (2022). Graph neural networks in recommender systems: a survey. *ACM Computing Surveys*, *55*(5), 1–37.

Wu, J., Wang, X., Feng, F., He, X., Chen, L., Lian, J., et al. (2021). Self-supervised graph learning for recommendation. In *Proceedings of the 44th international ACM SIGIR conference on research and development in information retrieval* (pp. 726–735).

Xia, L., Huang, C., Xu, Y., & Shi, J. (2023). Graph-less collaborative filtering. In *The web conference*.

Xia, L., Huang, C., Xu, Y., Zhao, J., Yin, D., & Huang, J. (2022). Hypergraph contrastive collaborative filtering. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 70–79).

Xia, H., Wang, Z., Du, B., Zhang, L., Chen, S., & Chun, G. (2019). Leveraging ratings and reviews with gating mechanism for recommendation. In *Proceedings of the 28th ACM international conference on information and knowledge management* (pp. 1573–1582).

Yang, Y., Huang, C., Xia, L., & Li, C. (2022). Knowledge graph contrastive learning for recommendation. In *Proceedings of the 45th international ACM SIGIR conference on research and development in information retrieval* (pp. 1434–1443).

Yu, J., Yin, H., Li, J., Wang, Q., Hung, N. Q. V., & Zhang, X. (2021). Self-supervised multi-channel hypergraph convolutional network for social recommendation. In *Proceedings of the web conference 2021* (pp. 413–424).

Zhang, Y., Lai, G., Zhang, M., Zhang, Y., Liu, Y., & Ma, S. (2014). Explicit factor models for explainable recommendation based on phrase-level sentiment analysis. In *Proceedings of the 37th international ACM SIGIR conference on research & development in information retrieval* (pp. 83–92).

Zhao, C., Li, C., Xiao, R., Deng, H., & Sun, A. (2020). CATN: Cross-domain recommendation for cold-start users via aspect transfer network. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval* (pp. 229–238).